



UNIVERSITÀ DI PISA

Distributed Systems and Middleware
Technologies

Project Specifications Proposal

Distributed Crazy Time

Academic Year 2025-2026

Authors: Gabriele Caioli, Klaudio Ciacia, Lorenzo Vanni

Contents

1	Project Specifications	2
1.1	Use Cases	2
1.2	Synchronization and Communication Issues	3
1.3	Design Ideas	4
1.3.1	System Architecture Diagram	5

1 Project Specifications

Distributed Crazy Time is a distributed, real-time betting web application inspired by popular live casino game shows.

Game Mechanics Overview: The core mechanic revolves around a large spinning wheel divided into multiple segments. Players place bets on where the wheel will stop within a strict time window. Most segments offer straightforward multiplier payouts (e.g., 1x, 2x, 5x, 10x), while special segments trigger interactive "mini-games" (such as *Pachinko*, *Coin Flip*, *Cash Hunt*, and *Crazy Time*). When a mini-game is triggered, the final payout is determined by additional random events within that specific sub-game.

A distributed backend engine ensures fair random number generation, manages user wallets securely, and synchronizes the game state across all active clients in real-time.



Figure 1: Example of the Crazy Time game interface: the main wheel and the betting grid.

1.1 Use Cases

An **Unregistered User** can:

- Register to the service to create a wallet.

An **Unlogged User** can:

- Login to the service as a Player.

A **Logged Player** can:

- Logout;
- View the live countdown timer and the current state of the wheel/mini-games;
- Place bets on one or more wheel segments (if the betting window is open);
- View their current wallet balance and betting history.

The **System** must:

- Maintain and update the Players' wallet balances securely;
- Synchronize the countdown timer and game status across all connected clients;
- Reject any bets placed after the "No more bets" signal;
- Generate a single, authoritative random outcome for each wheel spin and mini-game event;
- Gracefully recover from the crash of the node generating the game outcomes, ensuring the game continues without state corruption.

Live Game Status: MAIN WHEEL SPINNING...
Time left to bet: [CLOSED]

Your Balance: \$ 150.00
Your Bets: \$ 10 on [Pachinko], \$ 5 on [Number 10]

Waiting for outcome...

Figure 2: Mock-up of the Player's dashboard during a spin

1.2 Synchronization and Communication Issues

In this application, we will face the following synchronization and communication issues:

- **State Synchronization:** All client nodes must receive synchronized updates regarding the betting timer, the main wheel's outcome, and the active mini-game status to ensure fairness.
- **Leader Election for RNG:** The random outcome of the wheel must be generated by a single authoritative node (the "Dealer"). In case the current Dealer node crashes, the Erlang cluster must execute a Leader Election algorithm (e.g., Bully Algorithm or Ring Election) to elect a new Dealer and resume the game seamlessly (in that case we can consider that the system will give back the bets of each client).
- **Consistent State Cut (Snapshot):** When the timer expires ("No more bets"), the system must guarantee that no further bets are processed. A Snapshot algorithm (e.g., Chandy-Lamport) can be utilized to capture the consistent global state of all accepted bets across the worker nodes before resolving the payouts.
- **Distributed Concurrency:** The system must handle thousands of concurrent bet requests, ensuring that wallet deductions and bet registrations are processed safely without data races.

1.3 Design Ideas

Our proposal for the implementation of this system relies on a hybrid Java/Erlang architecture to exploit the strengths of both ecosystems:

- **Client Nodes (Frontend & Gateway):** The user interface will be developed in HTML/JS. The backend serving the clients will be a Java web server (e.g., hosted at 10.0.2.15 for our testing environment) using Jakarta EE Servlets or Spring Boot/Tomcat. This Java layer will act as an API Gateway, managing user authentication, persistent data storage for wallets (via JDBC/JPA), and maintaining real-time communication with the browsers (e.g., via WebSockets or HTTP long-polling).
- **Middleware Integration:** To decouple the web gateway from the game engine, communication between the Java nodes and the Erlang cluster will be handled via a Message Oriented Middleware (e.g., RabbitMQ via JMS).
- **Server Node (Erlang Game Engine):** The core game logic will run on a distributed Erlang cluster. We will leverage the Actor Model to map game entities to lightweight processes: there will be a main "Wheel Process", and independent processes for each distinct mini-game (e.g., "Pachinko Process", "Coin Flip Process", "Cash Hunt Process", and "Crazy Time Process").

During a game round, if the main wheel lands on a bonus segment, the Wheel Process will delegate the execution to the corresponding mini-game process. We will use OTP Supervision Trees to monitor these worker processes, ensuring that if a specific mini-game crashes due to an unexpected error, it is handled and restarted independently without bringing down the entire system. The Erlang nodes will also be responsible for running the Leader Election, managing the countdown timer, executing the Snapshot algorithm to lock the bets, and publishing the game outcomes back to the Java gateway.

1.3.1 System Architecture Diagram

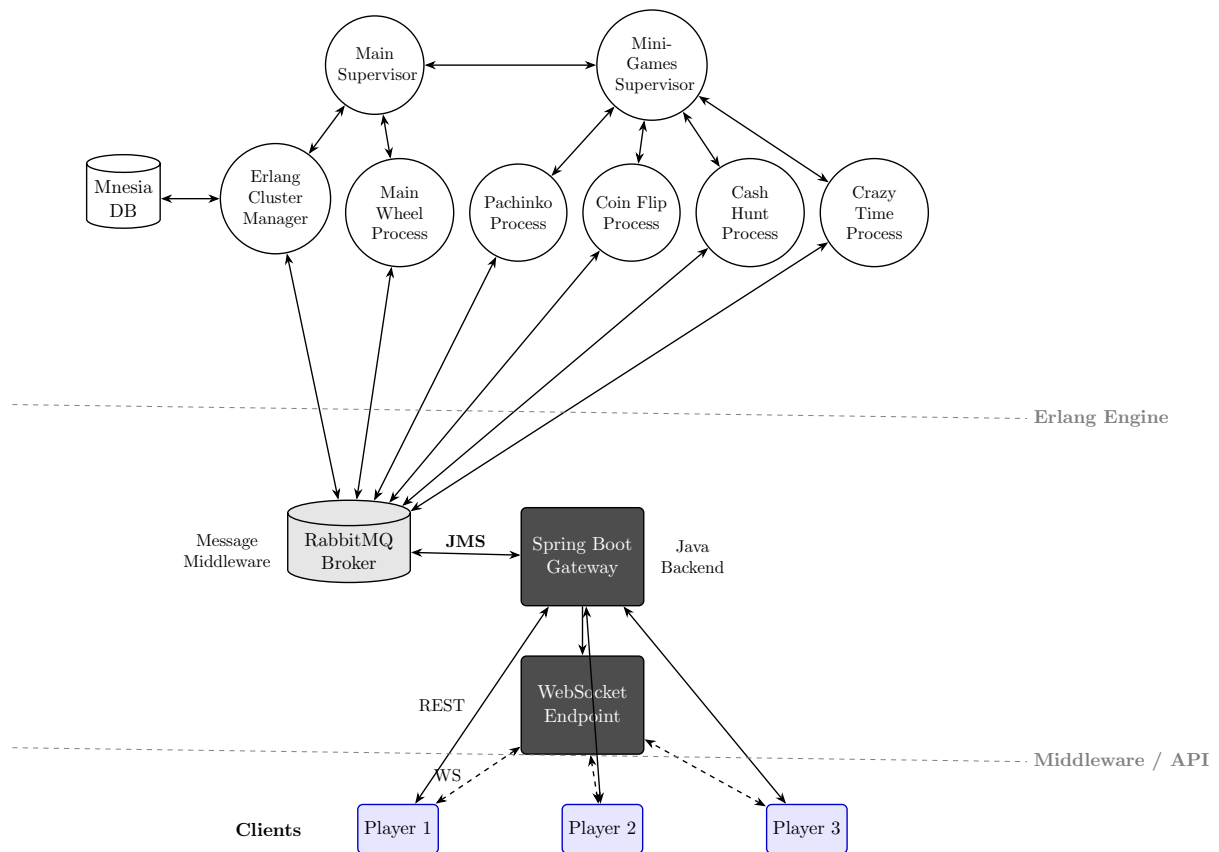


Figure 3: Architectural design of the Distributed Crazy Time platform, highlighting the OTP Supervision Tree, the RabbitMQ middleware, and the Spring Boot API Gateway.